

ASSIGNMENT - 4

Sub Code: CSA10

Subject Name: Software Engineering

No.	Questions	Marks	CO
1	Explain Mutation Testing and Fuzz Testing. How do these testing techniques enhance the reliability and security of software systems?	7	4
2	Describe the Key Stages of a CI/CD Pipeline (using Jenkins or GitHub Actions). How does using Docker for containerization support continuous delivery?	7	4
3	Case Study – Intermittent Slowdowns in a Large SaaS Application: Explain how monitoring and logging tools such as Prometheus, Grafana, and the ELK Stack can help detect, diagnose, and resolve performance issues.	6	4

1.

1. Mutation Testing and Fuzz Testing

Mutation Testing:

Mutation testing is a software testing technique in which small changes, called *mutations*, are intentionally introduced into the source code. The existing test cases are then executed to check whether they can detect these changes. If a test case detects the mutation, the mutant is considered **killed**. If not, the test suite may be weak.

Example:

Original code: if (a > b)

Mutated code: if (a < b)

If the test cases fail for the mutated code, they are effective in detecting errors.

Fuzz Testing:

Fuzz testing, or fuzzing, provides a program with large amounts of unexpected, random, or malformed input to identify crashes, vulnerabilities, memory errors, and unexpected behavior.

How they improve reliability and security:

- Mutation testing improves the quality and effectiveness of test cases.
- Fuzz testing identifies unexpected failures and security vulnerabilities.
- Both techniques help discover defects that normal testing may miss.

- They improve software reliability, robustness, and fault tolerance.
- Fuzzing is especially useful for detecting security issues such as crashes, buffer overflows, and invalid input handling.
- Together, they increase confidence that software behaves correctly under both normal and abnormal conditions.

2. Key Stages of a CI/CD Pipeline and Docker

A CI/CD pipeline automates the process of building, testing, and delivering software. Tools such as Jenkins and GitHub Actions can be used to implement it.

Key stages:

1. Source Code Management:

Developers push code to a Git repository such as GitHub.

2. Build:

The pipeline automatically compiles the application and installs required dependencies.

3. Automated Testing:

Unit tests, integration tests, security tests, and other automated tests are executed.

4. Code Quality and Security Checks:

Static code analysis and vulnerability scanning are performed to identify problems.

5. Package/Artifact Creation:

A deployable application package or Docker image is created.

6. Deployment:

The application is automatically deployed to staging or production environments after successful tests and approvals.

7. Monitoring and Feedback:

The deployed application is monitored for errors and performance issues. Feedback is used for future improvements.

Role of Docker in Continuous Delivery

Docker packages an application along with its dependencies and configuration into a container. This ensures that the application runs consistently across development, testing, CI servers, and production.

Docker supports continuous delivery by:

- Providing a consistent environment.
- Reducing dependency and configuration problems.
- Making deployment faster and repeatable.
- Allowing the same Docker image to be tested and deployed.
- Supporting easy rollback to an earlier image version.
- Making applications portable across different systems.

Thus, CI/CD automates software delivery, while Docker provides a consistent and portable environment for that delivery.

3. Case Study – Intermittent Slowdowns in a Large SaaS Application

- In a large SaaS application, intermittent slowdowns may be caused by high CPU usage, memory problems, database delays, network issues, or overloaded services. Monitoring and logging tools help identify the root cause.
- Prometheus collects and stores numerical metrics from applications and infrastructure.

It can monitor:

- CPU and memory usage
- Request rate
- Response time
- Error rate
- Database performance
- Service availability

For example, if response time increases whenever CPU usage becomes high, Prometheus can help identify this relationship.

Grafana

Grafana displays monitoring data from Prometheus in dashboards and graphs.

It helps teams:

- Visualize performance trends.
- Identify unusual spikes.
- Compare different services.
- Set alerts when performance crosses a threshold.

For example, Grafana can show that a particular microservice experiences high latency every day during peak traffic.

ELK Stack

The ELK Stack consists of:

- Elasticsearch – stores and searches logs.
- Logstash – collects and processes logs.
- Kibana – provides dashboards and visualizations for logs.

ELK helps developers investigate exactly what happened during a slowdown by searching application errors, database messages, timestamps, and request information.

Overall Process

The tools work together:

Prometheus → detects the performance problem

Grafana → visualizes and alerts on the problem

ELK Stack → examines logs to find the root cause

For example, if users report slow responses, Prometheus may detect increased latency. Grafana can show when and where the latency occurred, while ELK can reveal an error or database timeout responsible for the slowdown.

Developers can then fix the problem and use the same monitoring tools to verify that performance has improved.

Conclusion: These monitoring and logging tools provide continuous visibility into the SaaS application, allowing teams to detect, diagnose, resolve, and prevent performance problems, thereby improving reliability and user experience.

